Criterion B - Planning

Decomposition

1. **User Authentication and Security** (Maps to SC: Security and Privacy)

    - Hash passwords using werkzeug.security before storing in the database
    - Set a secret key in Flask to cryptographically sign session cookies
    - Define the parameterized queries through SQLAlchemy to prevent SQL injection attacks

2. **File Upload and Validation** (Maps to SC: File Upload, Error Handling)

    - Make a file upload component in React
    - Define the POST route in Flask
    - Validating that both the files are uploaded in PDF format
    - Display appropriate error messages for incorrect/invalid/corrupt file formats

3. **Data extraction and Structuring** (Maps to SC: Automatic Matching, Performance)

    - Using pdfplumber implement PDF table extraction
    - Create column mapping logic by using rapidfuzz
    - Implement date normalisation to standardise all date formats
    - Convert the extracted data into pandas dataframes for reconciliation

4. **Transaction matching and Tolerance handling** (Maps to SC: Tolerance, Automatic Matching)

    - Make the matching logic in pandas to compare rows from the two dataframes
    - Use rapidfuzz to implement description matching to handle slight naming differences
    - Create a tolerance input in react for the user to define date and amount difference thresholds
    - Add a status column to classify the matched rows based on tolerance applied

5. **Flagging Unmatched and irregular transactions** (Maps to SC: Flagging System)

    - Flag rows that failed the tolerance condition set by the user
    - Detect duplicate record within each dataframe
    - Identify the rows present in one dataframe but missing in other
    - Store all the flagged rows in a column with the respective reason

6. **Reconciliation report generation** (Maps to SC: Reconciliation Report)

    - Pass the classified dataframe (matched/unmatched status column) to reportlab to generate the PDF report
    - Create a Export button using react to download the reconciliation pdf

7. **User Interface and Progress Feedback** (Maps to SC: User Interface and Feedback)

    - Build upload, reconcile, export, history buttons and pages using react
    - Display the reconciliation table with colour coded rows (green, yellow, red) using Bootstrap
    - Display a progress bar in react while Flask is processing the PDFs
    - Show a summary card with all the matched and unmatched reconciliations

Planning

Week 1-2 Setup and Authentication

Dependencies - None

- Setup Flask, PostgreSQL, SQLAlchemy and React
- Build login UI
- Hash passwords

Week 3-4 File upload and PDF extraction

Dependencies - Authentication

- Make ‘upload’ UI component
- Define POST route
- Implement validation, pdfplumber extraction, column mapping and date normalisation

Week 5-6 Transaction Matching

Dependencies - Extraction

- Convert the extracted data into dataframes in pandas
- Create the row matching logic in pandas
- Implement description matching using rapidfuzz

Week 7-8 Tolerance, Flagging and Report Generation

Dependencies - Matching

- Create ‘tolerance’ input in react
- Create logic to flag unmatched, duplicated and missing rows and store the results
- Aggregate counts and PDF generation using reportlab
- Build ‘export’ UI component

Week 9-10 GUI

Dependencies - Flagging

- Create - pages, colour coded tables, progress bar and summary cards

Week 11-12 Testing and bug fixes

Dependencies - GUI

- Test every module
- Conduct user testing
- Fix bugs based on the user feedback